

LeetCode 269 — Alien Dictionary (Premium)

Problem Statement

Imagine Aadya comes from an alien planet.

Instead of English alphabetical order `a b c d e f ...` they have their own alphabet. You don't know its order.

But you are given a dictionary of words already sorted according to the alien language.

Example

words = `wrt`, `wrf`, `er`, `ett`, `rftt`

these words are already sorted. Find the order of alien characters.

Example: `wrt`, `wrf`

Compare character by character:

- `w == w`
- `r == r`
- `t != f`

The first difference tells us **t comes before f**. We will compare only adjacent words, as that is enough info to build our dependency graph.

Invalid Case

Suppose `abc` , `ab` .

Dictionary says `abc` comes before `ab` .

Impossible.

Because in every language `ab < abc` . Shorter prefix always comes first.

Return `""`

Overall Algorithm

Step 1 Collect every unique character.

Step 2 Compare adjacent words. Build graph.

Step 3 Run Kahn.

Step 4 If Kahn processes every character Return order. Else Cycle. Return `""`

Implementation Details

words = `ab` , `ac` .

Characters `a` , `b` , `c` . Only these three exist.

Letters like `d` , `e` , `f` should never be pushed into the queue.

So we'll maintain:

```
vector<int> exist(26,0);
```

Whenever we see `'a'` we do:

```
exist['a' - 'a'] = 1;
```

Later while pushing indegree 0 nodes instead of:

```
for(int i = 0; i < 26; i++){
    if(indeg[i] == 0){
        q.push(i);
    }
}
```

we write:

```
for(int i = 0; i < 26; i++){
    if(exist[i] && indeg[i] == 0){
        q.push(i);
    }
}
```

Other initializations:

```
vector<vector<char>> adj(26);
vector<int> indeg(26,0);
vector<int> exist(26,0);
```



```

class Solution {
public:
    string alienOrder(vector<string>& words) {

        // basic crux ye he ki
        // dictionary already sorted he
        // bas hume character ordering nikalni he
        // character ordering hi graph banayegi
        // phir normal kahns algorithm

        vector<vector<char>> adj(26);
        vector<int> indeg(26,0);
        vector<int> exist(26,0);

        for(auto &word : words){
            for(auto &ch : word){
                exist[ch-'a'] = 1;
            }
        }

        // comparing adjacent words only
        int n = words.size();

        for(int i = 0; i < n-1; i++){

            string a = words[i];
            string b = words[i+1];

            // abc
            // ab
            // impossible dictionary

            if(a.size() > b.size() && a.substr(0, b.size()) == b){
                return "";
            }

            // pehla mismatch letter order decide krega
            int len = min(a.size(), b.size());
            for(int j = 0; j < len; j++){

                if(a[j] != b[j]){
                    adj[a[j]-'a'].push_back(b[j]);
                    indeg[b[j]-'a']++;
                    break; // break krna hoga kyunki eg: abz
                                //                               aca      b->c ok but z->a
                                //                               not ok , hence sirf pehla mismatch see crow
                }
            }
        }
    }
};

```

```

    }
}

queue<char> q;
for(int i = 0; i < 26; i++){
    if(exist[i] && indeg[i] == 0){
        q.push(char(i+'a'));
    }
}

// kahns
string ans = "";
while(!q.empty()){
    char curr = q.front();
    q.pop();
    ans.push_back(curr);
    for(auto &it : adj[curr-'a']){
        indeg[it-'a']--;

        // ek prerequisite character place ho gaya
        // future character aur closer

        if(indeg[it-'a'] == 0){
            q.push(it);
        }
    }
}

int totalChars = 0;
for(int i = 0; i < 26; i++){
    if(exist[i]) totalChars++;
}

if(ans.size() != totalChars){
    return "";
}

return ans;
}

};

```